

Pontificia Universidad Católica de Chile
Instituto de Ingeniería Matemática y Computacional
IMT2112 - Algoritmos Paralelos en Computaci´on Cient´ifica

Tarea 1

25 de septiembre de 2026

2° semestre 2026 - Profesor Ignacio Labarca

Sebastián Jiménez Ojeda - 24214833

Preliminares

Esta tarea se ejecutó en dos computadores distintos con los siguientes especificaciones:

Computador 1 (PC1):

- Sistema operativo: EndeavourOS
- Procesador: 12 × AMD Ryzen 5 5500
- Memoria: 16 GiB de RAM (15,5 GiB usable)

Computador 2 (PC2):

- Sistema operativo: EndeavourOS
- Procesador: 8 × AMD Ryzen 3 5300U with Radeon Graphics
- Memoria: 8 GiB de RAM (5,6 GiB usable)

Respuestas

Ítem A

Con el objetivo de generar datos sintéticos se utilizó la API de NumPy de generación de números aleatorios con el uso de una semilla fija, que nos asegura que los datos son reproducibles. Con su uso, se muestrearon un vector y una matriz desde una distribución normal estándar obteniendo $\beta^* \in \mathbb{R}^{K+1}$ y $X_0 \in \mathbb{R}^{N \times K}$. Luego, se obtuvo $X \in \mathbb{R}^{N \times (K+1)}$ añadiendo una columna de 1s al comienzo y el vector $\mathbf{y} = X\beta^* + \mathbf{z} \in \mathbb{R}^N$, con $\mathbf{z} \sim \mathcal{N}_N(\mathbf{0}, I)$.

```
rng = np.random.default_rng(42)
beta_real = rng.standard_normal(K+1)
datos = rng.standard_normal(size=(N, K))
X = np.insert(datos, 0, 1, axis=1)
y = X @ beta_real + rng.standard_normal(N)
```

Ítem B

Con la librería estándar de Python `time` se midió solo el tiempo de ejecución del bootstrapping para regresión lineal en cada versión, omitiendo cualquier otro factor externo. En la primera iteración de las versiones `bs_numpy` resultó ser, por lo menos, 10 veces más eficientes que las otras dos versiones para distintos valores de p y en ambos PC.

Luego de algunas modificaciones, las únicas mejoras al código que resultaron ser relevantes fue la reducción de copias de memoria de la matriz X. En `bs_sklearn` se pasó a utilizar

```
rng = np.random.default_rng(random_seed)
indices = rng.choice(N, size=N, replace=True)
pesos = np.bincount(indices, minlength=N)
modelo = LinearRegression(fit_intercept=False)
modelo.fit(X, y, sample_weight=pesos)
```

donde el uso de `sample_weight` permite a `sklearn` usar la matriz original en vez de producir copias de ellas. El peor de los casos, cuando $p = 12$, se ocupaba

$$\text{Tamaño de } X \times 8 \text{ bytes} \times p = 100{,}000 \times 301 \times 8 \text{ bytes} \times 12 = 2{,}82 \text{ GB}$$

lo que a veces producía errores por falta de memoria al tener otros programas abiertos. También se implementó su respectiva versión en `bs_numpy.py` usando *broadcasting* con los pesos, así intercambiamos las copias en memoria por multiplicaciones sobre la matriz. De todas formas, el rendimiento en términos de tiempo es básicamente el mismo o marginalmente mejor.

Como alternativa también se probó el modelo `Ridge` con penalización 0 de `sklearn` que resuelve el mismo problema usando el solver cholesky en vez de SVD, resultando en los

mismos tiempos de `bs_numpy`, pues es en esencia la misma implementación. Aún así, se optó por `LinearRegression` en la versión final por ser lo explicitado en el enunciado.

Ítem C

Los intervalos de confianza bootstrap al 95 % fueron construidos descartando el 2.5 % inferior y superior de las estimación $\hat{\beta}^{(b)}$. Empíricamente se encontró que el porcentaje de los 301 verdaderos coeficientes β^* que cae dentro de los valores límites ronda el 95 % tal como se esperaba. Por esto, se puede asegurar que los resultados producidos por cada versión son correctos.

Respecto a la equivalencia de los resultados, tenemos que `bs_sklearn` y `bs_numpy` son numéricamente equivalentes (con posibles diferencias marginales debido a los algoritmos de resolución, `np.linalg.solve` vs SVD) gracias al uso una semilla global y el uso de `np.random.SeedSequence` que genera la misma secuencia de semillas que se utilizan para generar los índices de remuestreo. Por otro lado, `bs_auto` difiere numéricamente porque la semilla interna produce un remuestreo distinto al hecho manualmente. Sin embargo, los intervalos de confianza generados van a ser estadísticamente equivalentes siempre.

La reproducibilidad de los resultados depende de factores como: (1) La independencia de las ejecuciones y su orden, donde la reproducibilidad se logra por haber generado una secuencia estática de semillas que se inyecta directamente en cada tarea. (2) Hardware y librerías, al cambiar de computador con una arquitectura diferente, sistema operativo o distintas versiones de las librerías se pierde la reproducibilidad exacta.

```
semillas_base = np.random.SeedSequence(42).generate_state(B)

betas_bootstrap = Parallel(n_jobs=args.p)(
    delayed(ajustar_un_resample)(
        X, y, semillas_base[i]) for i in range(B)
)
```

Ítem D

En el caso de EndeavourOS (Linux) se utiliza la llamada al sistema `fork()`, la cual crea un proceso nuevo (hijo) duplicando el proceso actual (padre). Los procesos de los hijos no realizan copias profundas de la memoria del padre, en su lugar la técnica Copy-on-Write (CoW) que pospone la duplicación de los datos hasta que se intenta modificarlos. Así, cada proceso obtiene un puntero a las direcciones de memoria de X e y originales, y como los procesos solo las leen (dada la implementación), solo se mantiene un copia (la original) de ambas componentes, consiguiendo un *overhead* de memoria inicial muy bajo.

Por otro lado, para la creación de procesos se utilizó el *backend* por defecto de `joblib`, `Loky`, que es una versión más robusta de `multiprocessing`, diseñado para evitar fallos sus más comunes y catastróficos. En él, se utiliza `np.memmap` automáticamente con arreglos que

superan 1 MB guardando un respaldo en memoria compartida para que cada proceso lea de ahí en vez de copiar data para cada proceso.

Ítem E

Al ejecutar `bs_numpy` para distintos valores p sin controlar nada más (valores por defecto) se observó con el monitor del sistema, que en ambos computadores siempre se utilizaban todos cores lógicos disponibles para el sistema. Este fue un claro indicio de *oversubscription*, que se explica fácilmente al correr la función `threadpool_info()` de la librería `threadpoolctl`, la cual nos reporta el número de threads que utilizan las librerías en uso. Con ella, se observa que las librerías subyacentes de álgebra lineal (`openblas` para `Numpy` y `SciPy`) y de paralelismo (`openmp` para `sklearn`) están configuradas para utilizar todos los threads disponibles del sistema. En el caso del PC1 al lanzar p procesos con `joblib`, el total de hilos de ejecución activos es $p \times 12$. En el caso más extremo, si se lanzan $p = 12$ procesos, el sistema operativo tiene que gestionar 144 hilos sobre los 12 cores lógicos disponibles, lo genera un *oversubscription* y una sobrecarga por los cambios de contexto.

Para corregir el *oversubscription* debemos limitar el número de threads t que utilizan las librerías subyacentes para cumplir la restricción $p{\cdot}t \leq p_{\text{máx}}$. Con el uso de `threadpool_limits` de la librería `threadpoolctl` podemos lograr esto, manteniendo la restricción para ambos computadores, donde $p_{\text{máx}} = 12$ para PC1 y $p_{\text{máx}} = 8$ para PC2.

Ítem F

En la tabla 1 y figura 1 se presentas los resultados de ejecutar las tres versiones variando la cantidad de procesos p. Se puede apreciar cómo en ambos computadores `bs_numpy` fue la versión más rápida, mientras que las otras dos versiones mantienen tiempos similares, siendo casi 10 veces más lentas que `bs_numpy`. También notar que el computador 2 no pudo ejecutar `bs_auto` y `bs_sklearn` para $p = 6, 7, 8$ por problema de memoria pese a las mejoras descritas en el ítem (b). El problema de la *oversubscription* consume más memoria de la esperada en estos casos.

Computador 1				Computador 2			
p	`auto`	`sklearn`	`numpy`	p	`auto`	`sklearn`	`numpy`
1	69.781	61.966	7.96	1	87.368	88.342	17.893
2	51.06	51.548	6.976	2	74.311	75.654	12.752
3	49.926	52.169	6.179	3	73.493	71.626	13.199
4	54.183	53.928	5.868	4	78.668	78.218	14.195
5	55.619	56.574	5.682	5	78.648	77.892	13.617
6	57.597	56.778	5.807	6	Error	Error	13.121
7	59.496	59.577	5.079	7	Error	Error	12.104
8	60.346	60.259	5.141	8	Error	Error	13.286
9	61.639	60.405	5.261				
10	62.474	61.247	5.451				
11	62.503	62.142	5.637				
12	63.704	62.863	5.754				

Cuadro 1: Tiempos de ejecución de ambos computadores en segundos (s)

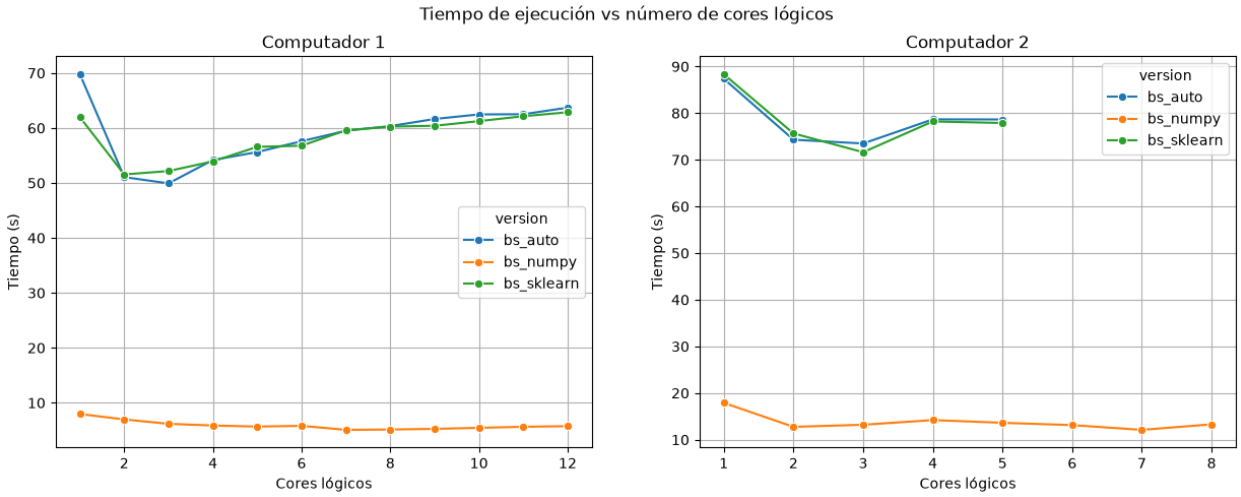


Figura 1: Tiempos de ejecución vs número de cores lógicos

Ítem G

Para este ítem se optó por calcular el speedup y eficiencia relativa, esto significa tomar $T(1)$ como el tiempo de ejecución con $p = 1$ de cada versión respectiva. Se usó esta definición exclusivamente para evaluar cómo escala el paralelismo de tareas junto al *overhead* y *oversubscription* de `joblib` (que se verá con más detalle en los ítem (h) e (i)) dentro de una misma implementación. Así también se aíslan las ineficiencias del código de las del sistema operativo.

En la figura 2 se aprecia cómo en todas las versiones del algoritmo, el speedup alcanza un valor máximo que luego descendiente lentamente. Esto se debe a que el *overhead* y el *oversubscription* comienza a afectar considerablemente la coordinación de los procesos sobre los recursos disponibles. Es fenómeno también se observa en la figura 3, que muestra la eficiencia de las versiones cayendo a medida que aumenta p, lo que indica que cada vez se aprovecha menos los recurso que se disponen.

En línea con lo anterior, se observa cómo las tres versiones se alejan fuertemente de la curva ideal tanto en speedup como en eficiencia. Los problemas con el *overhead* y el *oversubscription* son demasiado grandes para que las versiones sean cercanas a las curvas ideales.

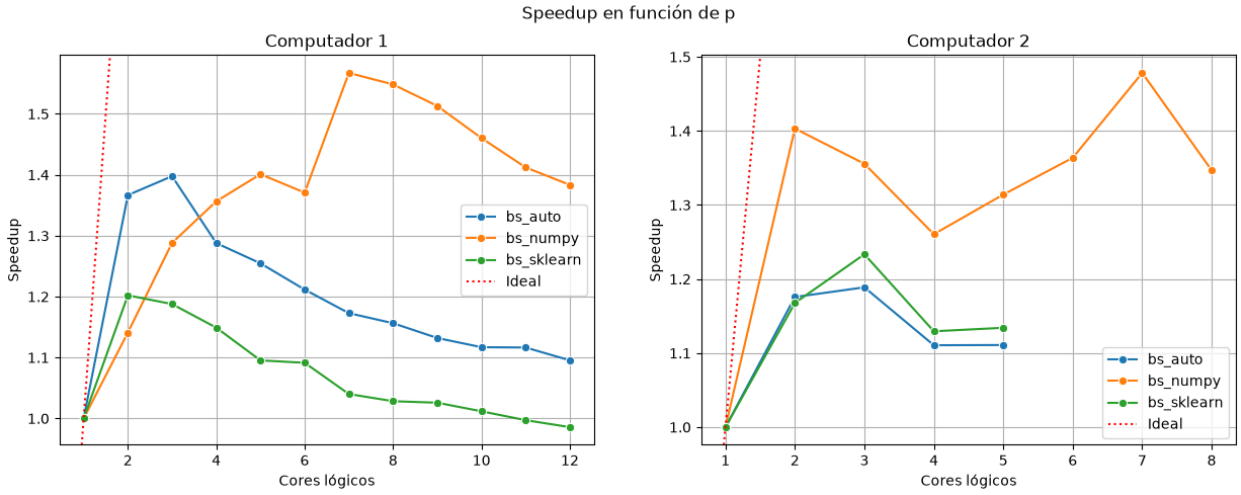


Figura 2: Speedup relativo en función de p

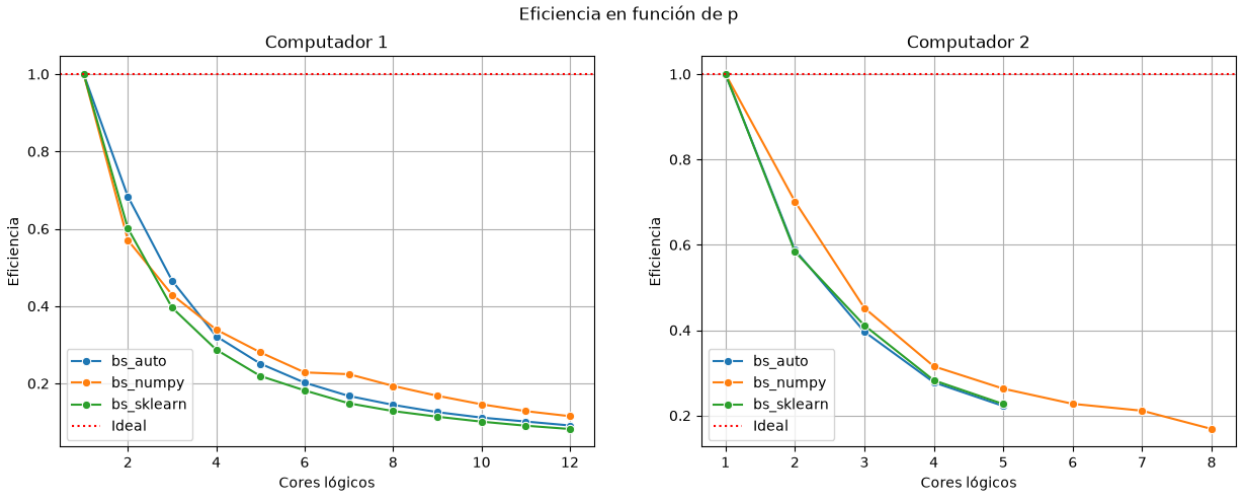


Figura 3: Eficiencia relativa en función de p

Hay que tener en cuenta que también era posible escoger $T(1)$ como el mejor (o peor) tiempo obtenido con $p = 1$. Con esto, los speedups de `bs_auto` y `bs_sklearn` sería ínfimos al compararlos con `bs_numpy`, demostrando cómo paralelizar un mal algoritmo secuencial no tiene sentido al existir uno superior.

Ítem H

En la tabla 4 se muestra como evoluciona el *overhead* de las tres versiones respecto al parámetro p. En ella se ve que en todas las versiones el overhead crece con p, aunque para la `bs_numpy` se mantiene más estable.

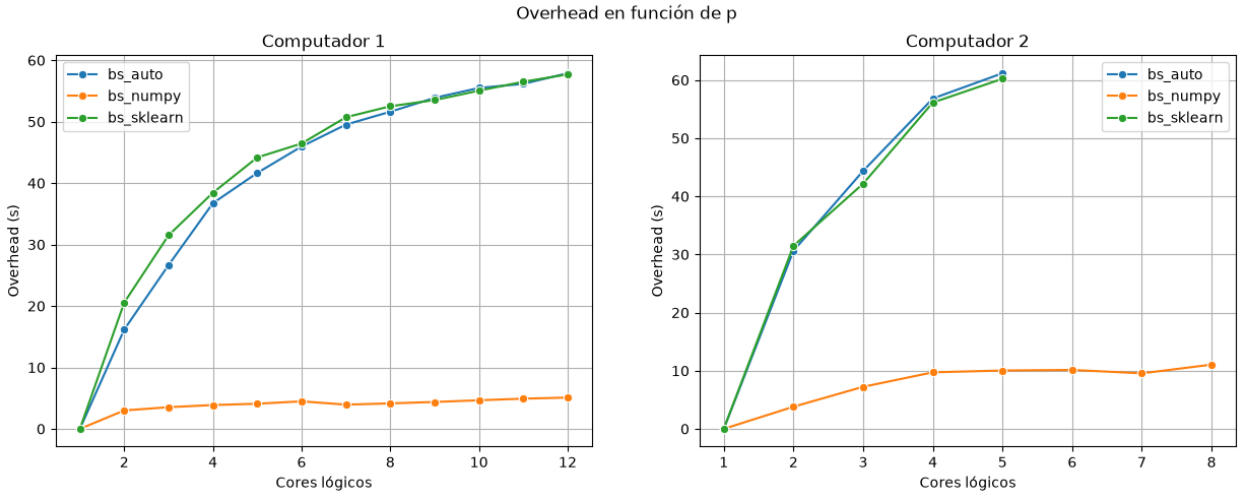


Figura 4: Overhead relativo en función de p

La primera y más relevante causa del *overhead* observado es la *oversubscription*. En las ejecuciones nunca se controló los threads utilizados por las librerías subyacentes, lo que produce un alto nivel de cambios de contexto en los núcleos. En el caso de `bs_numpy`, `OpenBLAS` intenta paralelizar las operaciones de multiplicación de matrices, por lo que al subir p, el sistema operativo interrumpe los procesos constantemente para alternar el uso de la CPU.

Segundo, al usar $B = 48$ resamples se produce un desbalance de carga cuando $p = 5, 7, 9, 10, 11$ pues ninguno es divisor del 48. Por lo que en estos casos $T(p)$ estará dictado por los hilos que tomen los resamples extras, se está sumando tiempo ocioso al overhead.

Tercero, el *backend* `loky` y el sistema operativo demoran un tiempo fijo en iniciar los procesos y mapear los arreglos a la memoria compartida. Si bien es tiempo es bastante pequeño, presenta una porción estadística más grande en `bs_numpy` ($T(1) = 7{,}96$s) comparada a `bs_auto` ($T(1) = 69{,}781$s).

Por último, en el computador 1 los threads 7 al 12 no son núcleo físico reales, sino hilos virtuales que comparte las unidades FPU y memoria cache L1 y L2 con los 6 núcleos físicos. Por lo que al superar $p = 6$, los procesos deben compartir estos recursos físicos, evitando que duplicar los hilos pueda cumplir con el speedup ideal.

Ítem I

En las figura 5 se muestran los resultados del computador 1. Hay un nuevo tiempo más bajo de 5.02s con la configuración $(p, t) = (8, 1)$, aunque las configuraciones híbridas $(4, 2)$ y $(5, 2)$ también presentas tiempos competitivos. De hecho, existe una varianza tal que estas y otras configuraciones con una cantidad de hilos cerca a la capacidad real del procesador a veces a aparecen como ganadoras. Las pequeñas variaciones en el planificador produce una varianza que hace que los resultados no sea deterministas.

En la misma figura, se observa lo descrito en el ítem anterior, al mirar $p = 1$ se observa como tiempo baja constantemente hasta $t = 6$, pero al pasar a $t = 7$ el tiempo se dispara. La arquitectura del procesador juega un papel importante aquí, el séptimo hilo tiene que compartir recursos físicos con otro hilo, introduciendo un *overhead* severo.

Por otro lado, la figura 6 muestra los resultados del computador 2. Aquí el ganador es $(p, t) = (1, 4)$, resulta mucho más eficiente paralelizar el trabajo interno de la librería `OpenBLAS` que dividir el trabajo en múltiples proceso de Python. La arquitectura del procesador nuevamente juega un papel importante en esto, el cambio de contexto y gestión de procesos independiente se penaliza más, favoreciendo un proceso pesado que aproveche la memoria compartida.

En general, no existe una configuración universal superior a otra, la elección más favorable depende del sistema que se esté utilizando en el momento. Los computadores utilizados resultaron tener configuraciones ganadoras opuestas, mientras uno tiene una mayor capacidad de multiprocesamiento con un bajo costo de cambios de contexto, el otro posee un entorno más restrictivo cuya mejor estrategia es utilizar las funciones optimizadas de librerías de bajo nivel sobre múltiples hilos.

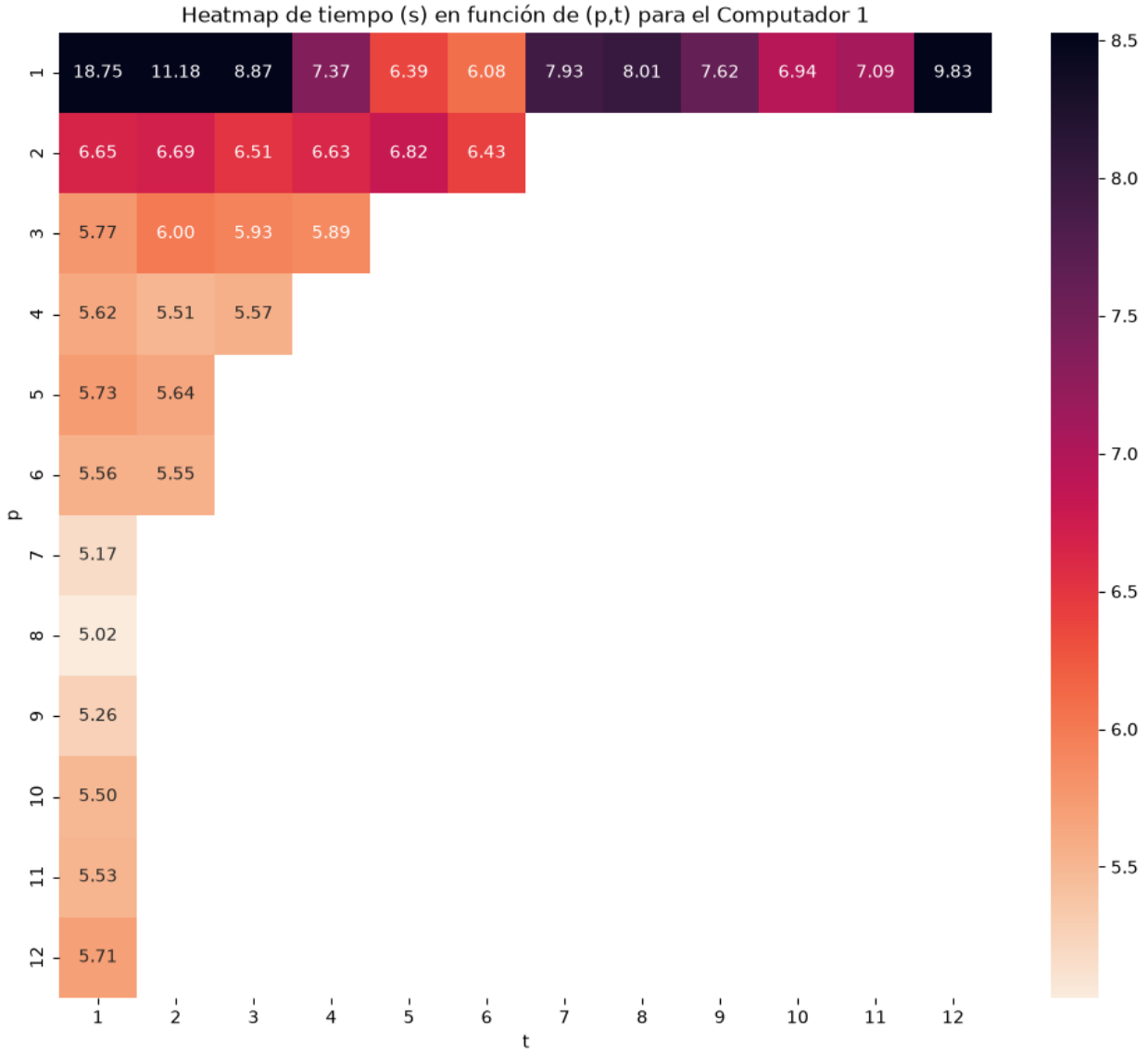


Figura 5: Tiempos (s) para los thread (p, t) en el computador 1

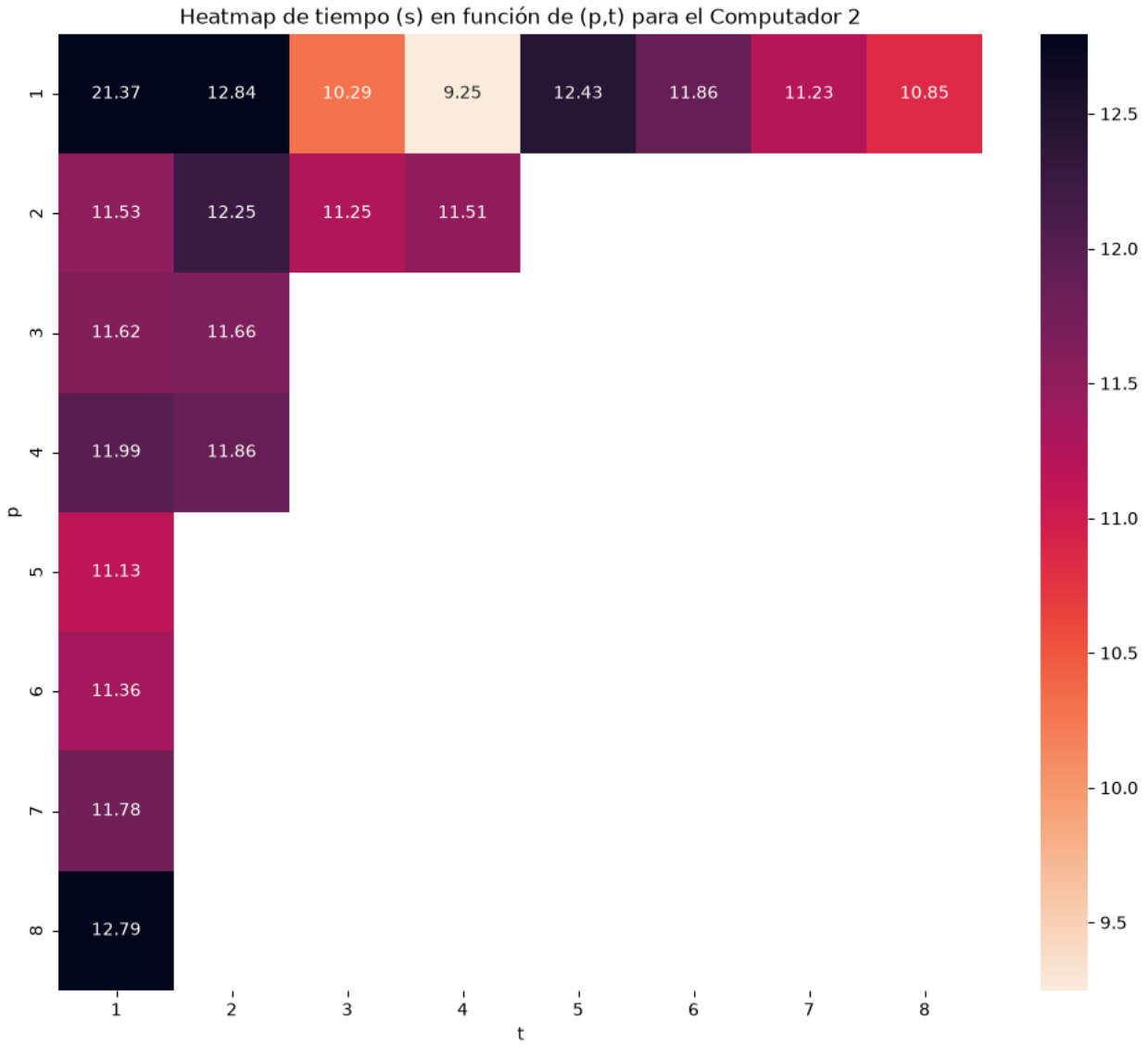


Figura 6: Tiempos (s) para los thread (p, t) en el computador 2

Ítem J

A lo largo del informe se realizaron varias comparaciones y observaciones sobre los resultados obtenidos, entre ellas se descubrieron tres grandes ejes que dejan ver las fortalezas y limitaciones de cada sistema:

1. Límites de RAM y *Overhead*: La más grande diferencia observada fue la interrupción abrupta de `bs_auto` y `bs_sklearn` debido a la falta de RAM en el PC2, causada por el *overhead* para valores $p \geq 6$. El sistema operativo mató los proceso para proteger el equipo ante el gran consumo de memoria, debido a que cada proceso mantenía un resample de 240

MB. Mientras que en el PC1, la holgura de RAM hizo posible completar sin complicaciones 12 procesos simultáneos. Esta situación muestra como la falta de recursos penaliza gravemente la paralelización de procesos incluso con optimizaciones de memoria.

2. Diferencia en las estrategias óptimas: En el ítem anterior se vio cómo la diferencia en arquitectura de la CPU afecta la elección de una configuración óptima. Los heatmaps revelaron comportamientos opuestos entre los computadores, uno favorece el paralelismo de procesos a nivel de sistema operativo y otro favorece el paralelismo de dato con los hilos de las librerías subyacentes.

3. Rendimiento base y núcleos virtuales: El procesador del computador 1 es simplemente mejor a nivel base, al comparar los tiempos secuenciales (del ítem (f)) vemos una diferencia significativa de 7.96s frente a 17.89s. Además, en el PC1 se pudo ver el cuello de botella del Simultaneous Multithreading (SMT) respecto de los núcleos virtuales con la drástica subida en tiempo de ejecución a usar el séptimo hilo.

Finalmente, se entiende que para paralelizar tareas no basta únicamente con un buen código o algoritmo, es necesario entender el sistema con el que se trabaja, respetando las capacidades dadas por los recursos disponibles.

Uso de IA

Para el desarrollo de esta tarea se utilizó Gemini 3.1 Pro como asistente interactivo para: el entendimiento detallado de los requerimientos del enunciado; la optimización algorítmica de las tres versiones del ítem (b) para reducir el consumo de RAM; la comprensión de conceptos de sistemas operativos vinculados a `joblib` (Copy-on-Write y memoria compartida); el cálculo de métricas de escalabilidad y la interpretación de los mapas de calor; el análisis de contención de recursos (oversubscription y SMT) en las distintas arquitecturas de los computadores; y orientación sobre la estructuración general del informe.